

Natural Language Processing

07. Mécanismes d'Attention et Transformers

Kenza BOUZIDI

`kenza.bouzidi.ext@u-pec.fr`

30 Janvier 2026

Plan du cours

- Introduction et Motivation
- Le Mécanisme d'Attention
- Self-Attention et Cross-Attention
- Multi-Head Attention
- Encodage Positionnel
- Architecture Complète du Transformer
- Composants Auxiliaires
- Analyse de Complexité
- Entraînement des Transformers
- Variantes Architecturales
- Interprétabilité
- Applications
- Conclusion

Introduction et Motivation

Problèmes majeurs des architectures récurrentes :

1. Traitement séquentiel

- Impossible de paralléliser l'entraînement
- Temps de calcul $\mathcal{O}(n)$ opérations séquentielles

Problèmes majeurs des architectures récurrentes :

1. Traitement séquentiel

- Impossible de paralléliser l'entraînement
- Temps de calcul $\mathcal{O}(n)$ opérations séquentielles

2. Dépendances longue distance

- Difficulté à capturer les relations entre tokens éloignés
- Information "diluée" au fil des pas de temps

Problèmes majeurs des architectures récurrentes :

1. Traitement séquentiel

- Impossible de paralléliser l'entraînement
- Temps de calcul $\mathcal{O}(n)$ opérations séquentielles

2. Dépendances longue distance

- Difficulté à capturer les relations entre tokens éloignés
- Information "diluée" au fil des pas de temps

3. Gradient vanishing/exploding

- Rétropropagation instable sur longues séquences
- LSTM/GRU : amélioration partielle seulement

Comparaison RNN vs Transformer

Critère	RNN	Transformer
Opérations séquentielles	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Complexité par couche	$\mathcal{O}(n \cdot d^2)$	$\mathcal{O}(n^2 \cdot d)$
Chemin max (2 tokens)	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Parallélisation	Non	Oui

Avantage clé

Le Transformer connecte directement chaque paire de tokens en $\mathcal{O}(1)$ couches !

"Attention Is All You Need"

Vaswani et al., NeurIPS 2017

Contributions principales :

- Suppression complète des récurrences
- Architecture basée uniquement sur l'attention
- Résultats SOTA en traduction (EN-DE, EN-FR)
- Entraînement beaucoup plus rapide

Impact

Base de BERT, GPT, T5, LLaMA, et tous les LLMs modernes

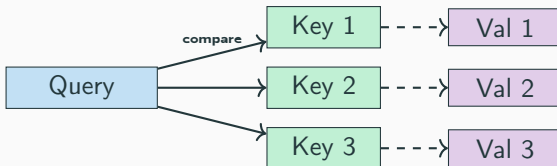
Le Mécanisme d'Attention

Intuition : L'attention comme recherche

Analogie

Chercher une information dans un livre :

- **Query (Q)** : Votre question ("Capitale de la France ?")
- **Key (K)** : Résumé de chaque phrase (index)
- **Value (V)** : Contenu de chaque phrase



Scaled Dot-Product Attention

Définition

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Dimensions :

- $Q \in \mathbb{R}^{n \times d_k}$: n requêtes de dimension d_k
- $K \in \mathbb{R}^{m \times d_k}$: m clés de dimension d_k
- $V \in \mathbb{R}^{m \times d_v}$: m valeurs de dimension d_v
- Sortie $\in \mathbb{R}^{n \times d_v}$

Étapes du calcul d'attention

1. Scores de similarité :

$$S = QK^T \in \mathbb{R}^{n \times m}$$

2. Normalisation :

$$S' = \frac{S}{\sqrt{d_k}}$$

3. Softmax (par ligne) :

$$\alpha_{ij} = \frac{\exp(S'_{ij})}{\sum_{l=1}^m \exp(S'_{il})}$$

4. Agrégation pondérée :

$$\text{Output}_i = \sum_{j=1}^m \alpha_{ij} V_j$$

Pourquoi diviser par $\sqrt{d_k}$?

Problème sans normalisation :

Si $q_i, k_j \sim \mathcal{N}(0, 1)$ i.i.d., alors :

$$\text{Var} \left(\sum_{l=1}^{d_k} q_l k_l \right) = d_k$$

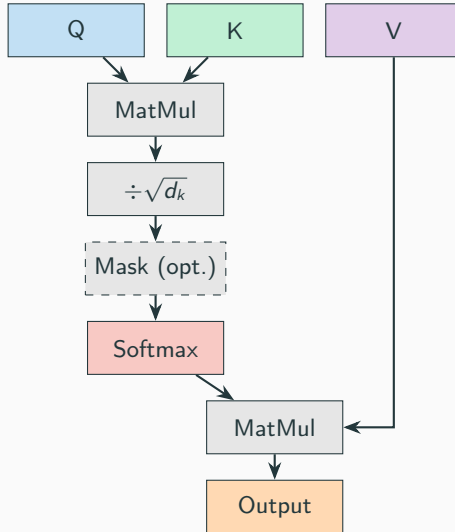
- Pour $d_k = 64$: variance = 64, écart-type ≈ 8
- Scores très grands $\Rightarrow$ softmax sature
- Gradients quasi-nuls !

Solution

Diviser par $\sqrt{d_k}$ ramène la variance à 1

$$\text{Var} \left(\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}} \right) = 1$$

Diagramme de l'attention



Exemple numérique (1/3)

Données : $n = 3$, $d_k = d_v = 4$

$$Q = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{pmatrix}$$

$$K = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

$$V = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{pmatrix}$$

Exemple numérique (2/3)

Étape 1 : QK^T

$$QK^T = \begin{pmatrix} 1 & 1 & 2 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \end{pmatrix}$$

Étape 2 : Division par $\sqrt{d_k} = 2$

$$\frac{QK^T}{2} = \begin{pmatrix} 0.5 & 0.5 & 1.0 \\ 0.5 & 0.5 & 0.5 \\ 0.5 & 0 & 0.5 \end{pmatrix}$$

Exemple numérique (3/3)

Étape 3 : Softmax (par ligne)

$$A = \begin{pmatrix} 0.245 & 0.245 & 0.510 \\ 0.333 & 0.333 & 0.333 \\ 0.378 & 0.244 & 0.378 \end{pmatrix}$$

Étape 4 : Sortie (1ère ligne)

$$\begin{aligned} \text{Out}_1 &= 0.245 \times (1, 2, 3, 4) + 0.245 \times (5, 6, 7, 8) \\ &\quad + 0.510 \times (9, 10, 11, 12) \\ &= (6.06, 7.06, 8.06, 9.06) \end{aligned}$$

Self-Attention et Cross-Attention

Self-Attention (Auto-Attention)

Définition

Q, K, V proviennent de la **même** séquence X :

$$Q = XW^Q$$

$$K = XW^K$$

$$V = XW^V$$

Exemple : *"Le chat qui dormait sur le canapé s'est réveillé"*

- Le verbe "réveillé" peut "regarder" le sujet "chat"
- Même s'ils sont séparés par plusieurs mots
- Chaque token interagit avec tous les autres

Définition

Q d'une séquence, K et V d'une **autre** séquence :

$$Q = X_{dec} W^Q \quad (\text{décodeur})$$

$$K = X_{enc} W^K \quad (\text{encodeur})$$

$$V = X_{enc} W^V \quad (\text{encodeur})$$

Usage :

- Traduction : le décodeur "regarde" l'encodeur
- Résumé : aligner sortie avec entrée
- Question-Answering : aligner question et contexte

Attention Masquée (Causale)

Problème : En génération, on ne doit pas voir le futur !

Solution : Masque

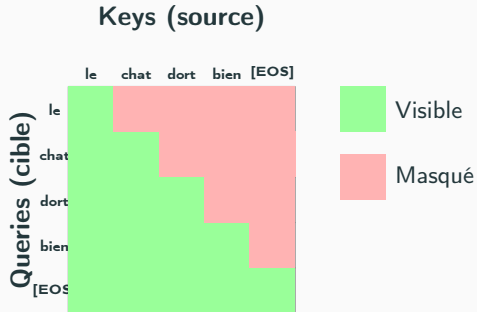
$$\text{Attention}_{\text{masked}}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V$$

Masque triangulaire :

$$M = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Après softmax : $e^{-\infty} = 0$ (pas d'attention sur le futur)

Visualisation du masque causal



Chaque token ne voit que lui-même et les précédents.

Multi-Head Attention

Problème d'une seule tête :

- Une seule "perspective" sur les données
- Difficile de capturer différents types de relations

Solution : Plusieurs têtes d'attention en parallèle

- Chaque tête peut se spécialiser
- Tête 1 : relations syntaxiques
- Tête 2 : positions adjacentes
- Tête 3 : coréférences
- etc.

Définition du Multi-Head Attention

Formule

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

où chaque tête :

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Dimensions :

- $W_i^Q, W_i^K \in \mathbb{R}^{d_{model} \times d_k}$
- $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$
- $W^O \in \mathbb{R}^{hd_v \times d_{model}}$
- Typiquement : $d_k = d_v = d_{model}/h$

Transformer original :

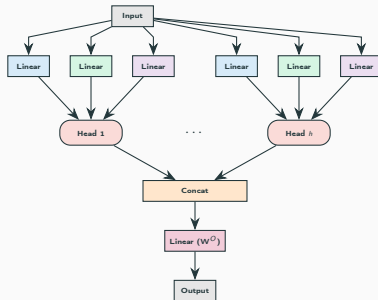
- $d_{model} = 512$
- $h = 8$ têtes
- $d_k = d_v = 512/8 = 64$

Coût computationnel

Même coût qu'une attention à tête unique avec dimension complète !

$$h \times \mathcal{O}(n^2 \cdot d_k) = \mathcal{O}(n^2 \cdot h \cdot d_k) = \mathcal{O}(n^2 \cdot d_{model})$$

Architecture Multi-Head



Observations sur modèles entraînés :

Tête	Pattern appris
1	Attention sur position précédente
2	Attention sur position suivante
3	Relation sujet-verbe
4	Relation déterminant-nom
5	Coréférence pronominale
6	Attention sur [SEP]/[CLS]

Remarque

Cette spécialisation émerge **naturellement** pendant l'entraînement !

Encodage Positionnel

Pourquoi encoder la position ?

Problème

L'attention est **invariante par permutation** !

Pour une permutation σ :

$$\text{Attention}(x_{\sigma(1)}, x_{\sigma(2)}, x_{\sigma(3)}) = \sigma(\text{Attention}(x_1, x_2, x_3))$$

Conséquence :

- "Le chat mange la souris"
- "La souris mange le chat"

⇒ Même représentation sans information de position !

Formules

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Propriétés :

1. Chaque position a un encodage unique
2. Généralise à des longueurs non vues
3. Relations relatives encodées linéairement

Usage :

$$X_{input} = E_{token} + E_{position}$$

Théorème

Il existe une matrice M_k (indépendante de pos) telle que :

$$PE_{pos+k} = M_k \cdot PE_{pos}$$

Preuve (idée) : Formules de somme trigonométriques

$$\begin{pmatrix} \sin(\omega(pos + k)) \\ \cos(\omega(pos + k)) \end{pmatrix} = \begin{pmatrix} \cos(\omega k) & \sin(\omega k) \\ -\sin(\omega k) & \cos(\omega k) \end{pmatrix} \begin{pmatrix} \sin(\omega \cdot pos) \\ \cos(\omega \cdot pos) \end{pmatrix}$$

⇒ Le modèle peut apprendre les relations positionnelles relatives

Autres encodages positionnels

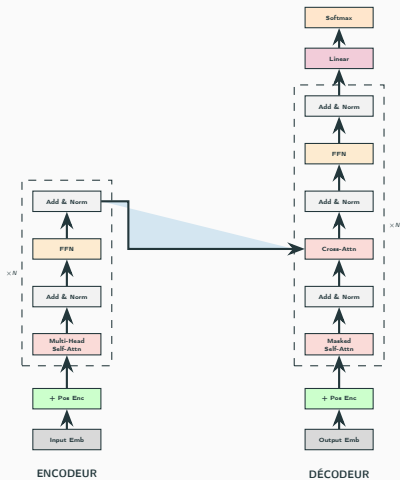
Type	Description
Sinusoidal	Formules sin/cos fixes
Appris	Embeddings appris (BERT, GPT)
Relatif	Encode distances relatives
RoPE	Rotation dans l'espace (LLaMA)
ALiBi	Biais linéaire sur attention

Tendance actuelle

RoPE et ALiBi dominant pour les LLMs car meilleure généralisation aux longues séquences

Architecture Complète du Transformer

Vue d'ensemble



Formules

$$Z_1 = \text{LayerNorm}(X + \text{MultiHead}(X, X, X))$$

$$Z_2 = \text{LayerNorm}(Z_1 + \text{FFN}(Z_1))$$

Composants :

1. Multi-Head Self-Attention
2. Connexion résiduelle + LayerNorm
3. Feed-Forward Network
4. Connexion résiduelle + LayerNorm

Configuration : $N = 6$ blocs empilés

Formules

$$Z_1 = \text{LayerNorm}(Y + \text{MultiHead}_{\text{masked}}(Y, Y, Y))$$

$$Z_2 = \text{LayerNorm}(Z_1 + \text{MultiHead}(Z_1, H_{\text{enc}}, H_{\text{enc}}))$$

$$Z_3 = \text{LayerNorm}(Z_2 + \text{FFN}(Z_2))$$

Différences avec l'encodeur :

1. Self-attention **masquée** (causale)
2. Couche de **cross-attention** supplémentaire
3. K, V viennent de l'encodeur dans la cross-attention

Composants Auxiliaires

Feed-Forward Network (FFN)

Définition

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

Dimensions :

- $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$
- $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$
- $d_{\text{ff}} = 4 \times d_{\text{model}} = 2048$ (original)

Position-wise : Appliqué indépendamment à chaque position

Variantes modernes : GELU, SwiGLU, GeGLU

Définition

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sigma + \epsilon} + \beta$$

Avec :

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

Pourquoi LayerNorm (pas BatchNorm) ?

- Indépendant de la taille du batch
- Fonctionne bien en inférence
- Normalise sur les features (pas le batch)

Formule

$$y = x + f(x)$$

Avantages :

1. Optimisation facilitée

- Gradient peut "court-circuiter" les couches
- $\frac{\partial y}{\partial x} = 1 + \frac{\partial f}{\partial x}$

2. Réseaux plus profonds

- Sans elles : instabilité au-delà de quelques couches
- Avec : 12, 24, 96+ couches possibles

3. Préservation de l'information

- L'identité est toujours une option

Post-Norm (Original) :

$$y = \text{LayerNorm}(x + f(x))$$

- BERT original
- Meilleure perf finale
- Moins stable

Pre-Norm (Moderne) :

$$y = x + f(\text{LayerNorm}(x))$$

- GPT-2, LLaMA
- Plus stable
- Pas besoin de warm-up

Tendance

Pre-Norm domine pour les grands modèles (stabilité cruciale)

Analyse de Complexité

Complexité de l'attention

Pour une séquence de longueur n et dimension d :

Opération	Complexité
QK^T	$\mathcal{O}(n^2 d)$
Softmax	$\mathcal{O}(n^2)$
Multiplication par V	$\mathcal{O}(n^2 d)$
Total	$\mathcal{O}(n^2 d)$

Goulot d'étranglement

Mémoire : Matrice d'attention $A \in \mathbb{R}^{n \times n} \Rightarrow \mathcal{O}(n^2)$

Comparaison des architectures

	Complexité	Ops séq.	Chemin max
Self-Attention	$\mathcal{O}(n^2 d)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
RNN	$\mathcal{O}(nd^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
CNN	$\mathcal{O}(knd^2)$	$\mathcal{O}(1)$	$\mathcal{O}(\log_k n)$

Trade-off :

- $n < d$: Attention plus efficace
- $n > d$: RNN plus efficace (mais séquentiel)
- En pratique : $n \approx 512 - 4096$, $d \approx 512 - 4096$

1. **Sparse Attention** (Longformer, BigBird)

- Attention locale + globale sur certains tokens
- Complexité : $\mathcal{O}(n \cdot k)$ avec k voisins

2. **Linear Attention** (Performer, Linear Transformer)

- Approximation kernel du softmax
- Complexité : $\mathcal{O}(nd^2)$

3. **Flash Attention** (Dao et al., 2022)

- Même calcul, meilleure utilisation mémoire
- Tiling pour rester dans le cache GPU
- 2-4x plus rapide en pratique

Entraînement des Transformers

Language Modeling (GPT-style) :

$$\mathcal{L}_{LM} = - \sum_{t=1}^T \log P(x_t | x_{<t}; \theta)$$

Masked Language Modeling (BERT-style) :

$$\mathcal{L}_{MLM} = - \sum_{t \in \mathcal{M}} \log P(x_t | x_{\setminus \mathcal{M}}; \theta)$$

où $\mathcal{M}$ = positions masquées (15% des tokens)

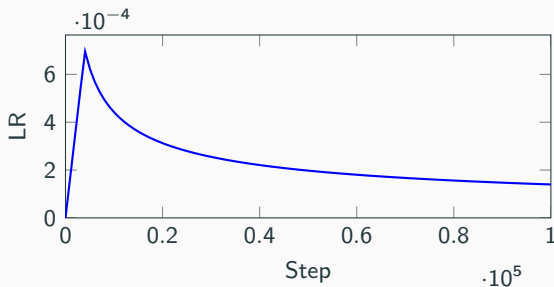
Sequence-to-Sequence :

$$\mathcal{L}_{S2S} = - \sum_{t=1}^T \log P(y_t | y_{<t}, x; \theta)$$

Learning Rate Schedule

Formule (original)

$$lr = d_{model}^{-0.5} \cdot \min(step^{-0.5}, step \cdot warmup^{-1.5})$$



- Warm-up : augmentation linéaire (4000 steps)
- Decay : décroissance en $1/\sqrt{step}$

Techniques utilisées :

1. Dropout ($p = 0.1$)

- Sur les poids d'attention
- Après le FFN
- Sur les embeddings

2. Label Smoothing ($\epsilon = 0.1$)

$$y'_i = (1 - \epsilon)y_i + \epsilon/K$$

3. Weight Decay

- Régularisation L2 : $\lambda \|\theta\|^2$

Variantes Arquitecturales

BERT (Devlin et al., 2019)

Bidirectional Encoder Representations from Transformers

Architecture :

- Encodeur uniquement
- Attention bidirectionnelle
- $N = 12$ ou 24 couches

Pré-entraînement :

- MLM (15% masqués)
- Next Sentence Prediction

Applications : Classification, NER, QA, similarité

GPT (Radford et al., 2018-2023)

Generative Pre-trained Transformer

Architecture :

- Décodeur uniquement
- Attention causale
- Jusqu'à 175B params (GPT-3)

Pré-entraînement :

- Language Modeling
- Auto-régressif

Applications : Génération, chat, code, raisonnement

Encoder-Decoder : T5, BART

T5 (Google)

- "Text-to-Text"
- Toutes tâches = génération
- Span corruption

BART (Meta)

- Denoising autoencoder
- Corruptions variées
- Fort en résumé

Modèle	Architecture	Attention
BERT	Encoder	Bidirectionnelle
GPT	Decoder	Causale
T5/BART	Enc-Dec	Mixte
LLaMA	Decoder	Causale + RoPE

Interprétabilité

Patterns typiques observés :



Diagonal



Vertical



Block

- **Diagonal** : attention sur position actuelle
- **Vertical** : attention sur token spécial ([CLS])
- **Block** : attention par segments

Observations empiriques :

1. Couches inférieures

- Information de surface
- N-grams, fréquences

2. Couches intermédiaires

- Syntaxe (arbres de dépendance)
- POS tagging émerge

3. Couches supérieures

- Sémantique
- Spécialisation pour la tâche

Référence

Clark et al. (2019) "What Does BERT Look At?"

Applications

Tâche	Modèles
Traduction	Transformer, mBART, NLLB
Résumé	BART, PEGASUS, T5
QA	BERT, RoBERTa, DeBERTa
Génération	GPT-4, Claude, LLaMA
Classification	BERT, DistilBERT
NER	BERT-CRF, SpaCy

Vision :

- ViT (Vision Transformer)
- CLIP (vision + langage)
- DALL-E, Stable Diffusion

Audio :

- Whisper (transcription)
- MusicGen (génération musicale)

Science :

- AlphaFold 2 (structure des protéines)
- Attention sur les résidus

Multimodal :

- GPT-4V, Gemini, Claude (vision + texte)

Conclusion

1. **Self-Attention** : capture les dépendances en $\mathcal{O}(1)$ couches
2. **Multi-Head** : plusieurs perspectives simultanées
3. **Positional Encoding** : injecte l'information d'ordre
4. **Architecture modulaire** : encodeur et/ou décodeur
5. **Parallélisation** : entraînement beaucoup plus rapide que RNN
6. **Scalabilité** : performances $\propto$ taille du modèle

Limitations

- Complexité quadratique $\mathcal{O}(n^2)$
- Coût énergétique de l'entraînement
- Interprétabilité des grands modèles
- Hallucinations et fiabilité

Directions de recherche :

- Attention efficace (Flash, Linear)
- Architectures alternatives (Mamba, RWKV)
- Alignement et sécurité
- Modèles multimodaux

1. Vaswani et al. (2017). *Attention Is All You Need*. NeurIPS.
2. Devlin et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers*. NAACL.
3. Radford et al. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI.
4. Brown et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS.
5. Dao et al. (2022). *FlashAttention*. NeurIPS.
6. Touvron et al. (2023). *LLaMA*. arXiv.

Questions ?